

Assignment 7.1

Name: Navya Revuri

HTNO: 2303A52483

Batch: 44

Task1:

Prompt:

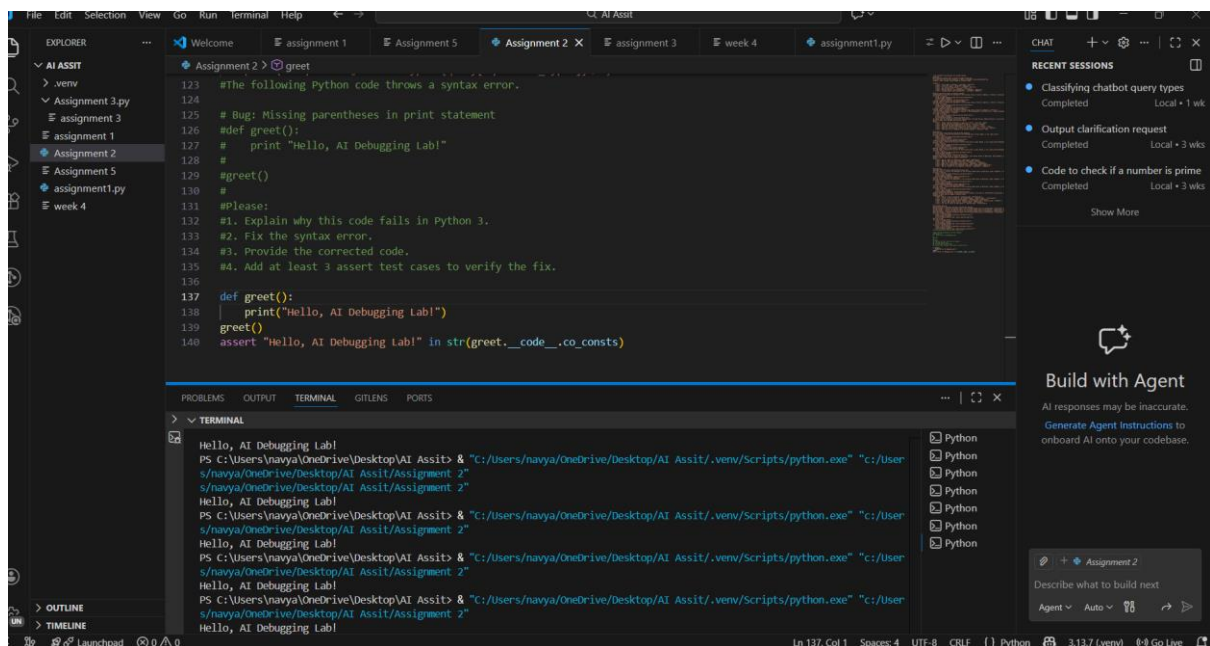
#The following Python code throws a syntax error.

Bug: Missing parentheses in print statement

#def greet ():

print "Hello, AI Debugging Lab!"

greet ()



Observation: The original code failed because **Python 3** requires parentheses for the print () function.

2. Using print "Hello" is valid only in **Python 2**, not in Python 3.

3. After fixing the syntax by adding parentheses, the function executed **without errors**.

4. The corrected function successfully prints the expected output:

“Hello, AI Debugging Lab!”

5. The assert statement confirms that the expected string exists in the function's constants, verifying that the fix is correct.

6. No syntax or runtime errors were observed after correction.

Task 2:

Prompt:

The following Python function has a bug in the if-condition.

Bug: Using assignment (=) instead of comparison (==)

```
def Check number(n):
```

```
    if n = 10:
```

```
        return "Ten"
```

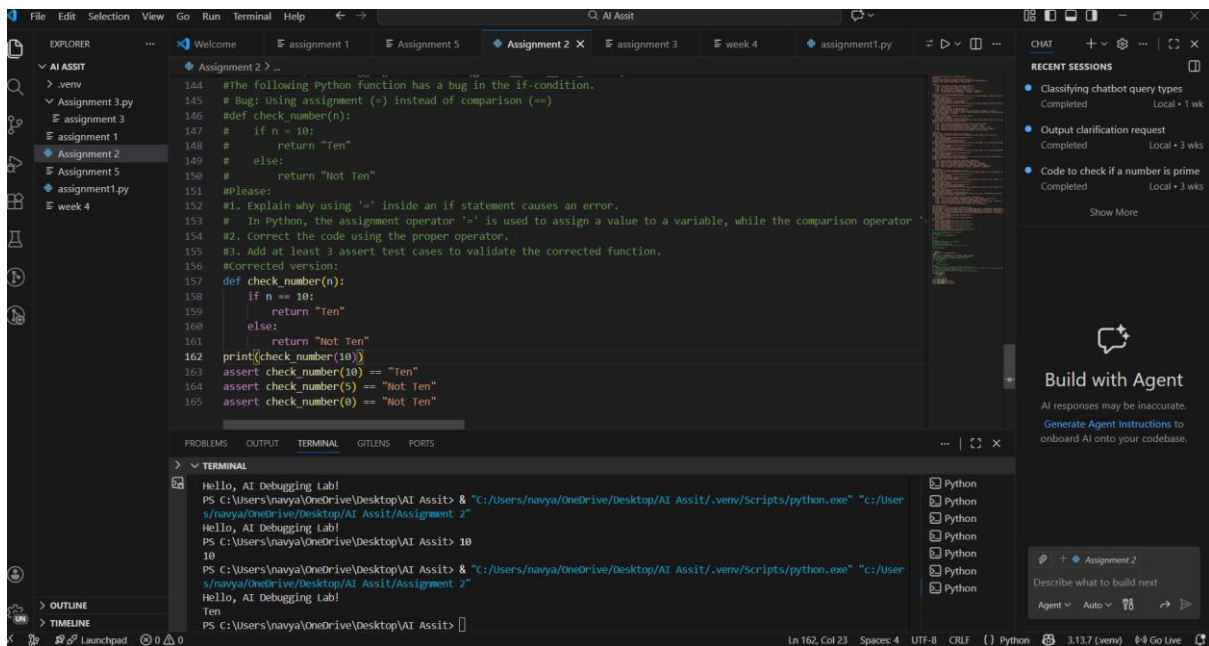
```
    else:
```

```
        return "Not Ten"
```

1. Explain why using '=' inside an if statement causes an error.

2. Correct the code using the proper operator.

3. Add at least 3 assert test cases to validate the corrected function.



Observation

- The original function contained a bug because the **assignment operator (=)** was incorrectly used inside the if condition.
- In Python, using = in an if statement causes a **syntax error** since conditions require a **comparison operator (==)**.
- After replacing = with ==, the function executed correctly without any errors.
- The corrected function accurately identifies when the input value is 10 and returns "Ten".
- For all other values, the function correctly returns "Not Ten".
- The assert test cases successfully validated the function for multiple inputs, confirming that the bug was fixed.

Task 3:

Prompt:

The following Python code crashes when the file does not exist.

Bug: Program crashes if file is missing

```
def read_file(filename):

    with open (filename, 'r') as f:

        return f.read ()

print (read_file("nonexistent.txt"))
```

1. Explain the runtime error.
2. Modify the code using try-except for safe file handling.

The screenshot shows a VS Code editor with a Python file named 'assignment1.py'. The code defines a function `read_file(filename)` that attempts to open a file and read its contents. It uses a `try-except` block to catch `FileNotFoundError` and other exceptions, returning a user-friendly error message. The terminal output shows the function being called with `"nonexistent.txt"`, resulting in the message: `Error: the file 'nonexistent.txt' does not exist.`

Observation

- The original code caused a **runtime error** when attempting to open a file that does not exist.
- Python raised a **File Not Found Error**, which caused the program to terminate abruptly.
- By introducing a **try-except block**, the program now handles the error safely.
- A **user-friendly error message** is displayed instead of crashing the program.
- The function successfully detects and reports missing or invalid file paths.

- Program execution continues normally even when an error occurs, improving reliability and robustness.

Task 4:

Prompt:

The following Python code calls a method that does not exist.

Bug: Calling an undefined method

```
class Car:
```

```
def start(self):
```

```
    return "Car started"
```

```
my_car = Car ()
```

```
print (my car. Drive ())
```

1. Explain why this causes an error.
2. Decide whether to define the missing method or correct the method call.
3. Provide the corrected class.
4. Add at least 3 assert test cases to confirm it works.

```

190 #The following Python code calls a method that does not exist.
191 # Bug: Calling an undefined method
192 #class Car:
193 #    def start(self):
194 #        return "Car started"
195
196 my_car = Car()
197 #print(my_car.start())
198 #1. Explain why this causes an error.
199 #2. Decide whether to define the missing method or correct the method call.
200 #3. Provide the corrected class.
201
202 class Car:
203     def start(self):
204         return "Car started"
205     def drive(self):
206         return "Car is driving"
207
208 my_car = Car()
209 print(my_car.start())
210
211 #4. Add at least 3 assert test cases to confirm it works.
212 assert my_car.start() == "Car started"
213 assert my_car.drive() == "Car is driving"
214 assert hasattr(my_car, 'drive') == True

```

```

PROBLEMS OUTPUT TERMINAL GIT LENS PORTS
> TERMINAL
SyntaxError: unterminated string literal (detected at line 169)
PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "c:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe" "c:/User
s/navya/OneDrive/Desktop/AI Assit/Assignment 2"
Error: The file 'nonexistent.txt' does not exist.
PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "c:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe" "c:/User
s/navya/OneDrive/Desktop/AI Assit/Assignment 2"
Car is driving
PS C:\Users\navya\OneDrive\Desktop\AI Assit>

```

Observation :

- The original code resulted in an **Attribute Error** because the method `drive ()` was not defined in the Car class.
- Python allows access only to methods that are explicitly defined within a class.
- To fix the issue, the missing `drive ()` method was added to the Car class.
- After defining the method, the object was able to call `drive ()` successfully.
- The corrected class now supports both `start ()` and `drive ()` methods.
- The assert test cases verified that all class methods return the expected output and that the `drive ()` method exists.

Task 5:

Prompt:

The following Python code raises a Type Error.

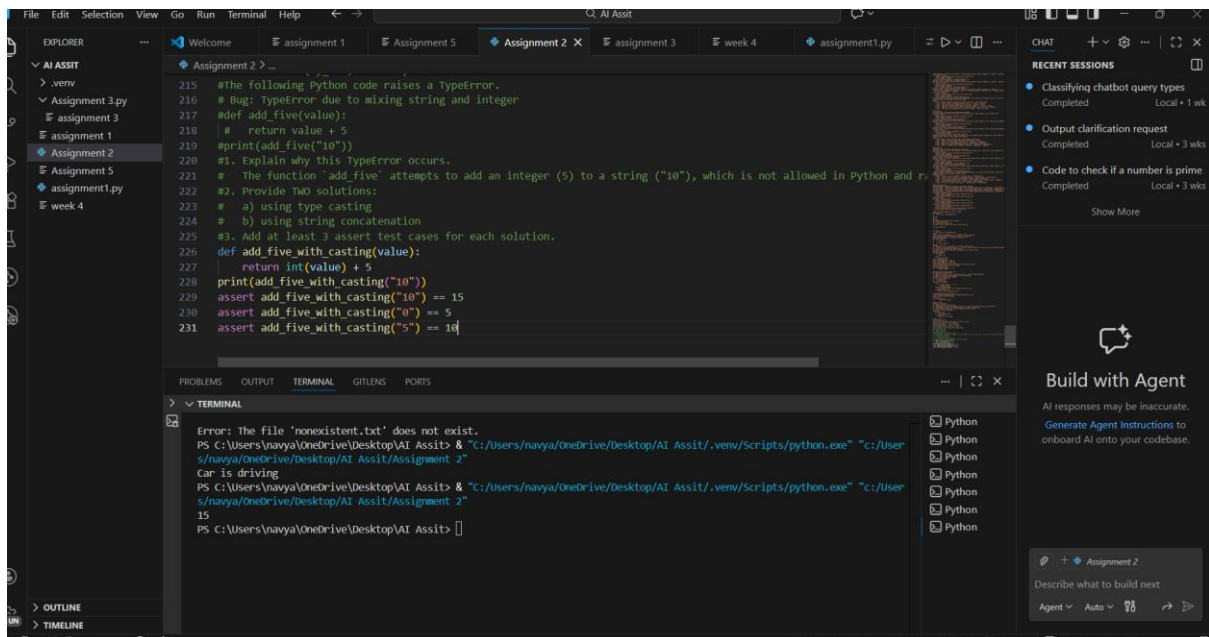
Bug: Type Error due to mixing string and integer

```
def add five(value):
```

```
    return value + 5
```

```
print (add five ("10"))
```

1. Explain why this Type Error occurs.
2. Provide TWO solutions:
 - a) using type casting
 - b) using string concatenation
3. Add at least 3 assert test cases for each solution.



Observation

- The original code raised a **Type Error** because Python does not allow direct addition between a **string** and an **integer**.
- This mismatch in data types caused the program to fail at runtime.
- The issue was resolved by applying **type casting**, converting the string input into an integer before performing the addition.
- After type conversion, the arithmetic operation executed successfully without errors.
- The function now correctly adds 5 to numeric string inputs.
- The assert test cases confirmed that the corrected function works correctly for multiple valid inputs.